

“Music Recommendation System”

ABSTRACT

According to Nielsen's Music 360 2014 study, 93% of the U.S. population listens to music, spending more than 25 hours each week jamming out to their favorite tunes [1]. Recommender systems have taken the entertainment and e-commerce industries by storm. Amazon, Netflix, and Spotify are great examples.

In this project, we have designed, implemented and analyzed song recommendation systems using various algorithms. Music recommendation is a very difficult problem as we have to structure music in a way that we recommend the favorite songs to users which is never a definite prediction. It is dynamic and sometimes influenced by factors other than users' or songs' listening history. We have used FMA Dataset [2], a freely-available collection of audio features and metadata for a million contemporary popular music tracks provided by Echo Nest, to find the correlations between users and songs and to learn from the previous listening history of users to provide recommendations for songs which users would prefer to listen most.

Everybody is using recommender systems nowadays in various forms and day to day these are getting improved because researchers are researching on making them better and better each day due to high competition in the market for giving better services and attracting customers. This project mainly focuses on only suggestion music for music lovers to help them listen to the songs those they may like. This framework enables clients to find new collections or tunes making the melodic list accessible for tuning in. Along these lines the framework can evaluate what artist or gathering would coordinate client inclinations to the client. For music enthusiastic people music is life and music is a larger part of everybody's lives because everything in this universe can be correlated to frequency and vibrations. Music helps us tuning ourselves with the universe and best thing about music is that nothing can relax you more than a pleasing melody. Due to all the good things about music and high demand of recommendersystems in the market we have chosen to do this project

TABLE OF CONTENTS

1 Introduction	1
2 Problem Statement	2
3 Literature Review	3
4 Related Work	4
5 Design (Include Methodology)	5-6
6 Implementation	7-10
6.1 Output Screen	
6.2 Result Analysis	
8 Conclusion and Future Scope	11
9 PROJECT CODE	12-63
References	64

1.

INTRODUCTION

Music recommender system is a system which learns from the user's past listening history and recommends them songs which they would probably like to hear in future. We have implemented Various algorithms to try to build an effective recommender system. We firstly implemented k-means model which was quite simple and intuitive. The quantity of songs accessible surpasses the listening limit of a person in the course of their life. It is repetitive for a person to in some cases browse a huge number of songs and there is additionally a decent possibility of passing up songs which could have been the well-suited for the event. There are two main types of recommendation systems: Collaborative filters, Content based filters

Content-based methods give recommendations based on the similarity of two song contents or attributes while collaborative methods make a prediction on possible preferences using a matrix with ratings on different songs.

Current recommendation systems are majorly based on the collaborative filters. These frameworks experience cold start problem; that is they break down when no chronicled information is accessible and therefore they cannot suggest another or less well known at this point comparable, melody to the client.

We will be using a content based filtering approach in this project. This framework will use Deep learning, feed forward neural network augmented by k-means classifier. The k-means classifier applies to the train data with the number of classes as the number of clusters to be made.

The dataset which will be used in this project is FMA Dataset

2.

RELATED WORK

The existing recommender systems using collaborative filtering algorithms have gained a great success. Netflix opened a challenge for the best collaborative filtering algorithm [3], and the winning algorithm using latent factor models could make 10.09% improvements over the algorithm used by Netflix at that time. Amazon uses user-user based and item-item based collaborative filtering [4], which greatly contributes to the success of the business. Recently a newer algorithm using neural network, neural collaborative filtering (He 2017) [5], was proposed.

For content-based algorithm, a lot of researchers have proposed different methods using Machine Learning technique, such as Decision Tree based [6], Support Vector Machine based [7], and even logistic regression [8]. We can fully utilize the knowledge we learnt from the class to implement these algorithms.

Music recommendation system shares some similarities with other commercial recommendation systems, but it focuses more on providing good and personalized advice on music, rather than goods for users to buy. The ideal music recommender system should be able to automatically recommend personalized music to human listeners. Different from books or movies, the length of a piece of music is much shorter, and the times that listening their favorite songs are normally more than once, which are the main difficulties we are going to face in this project.

3.

PROBLEM STATEMENT

In [2] a research paper by Song Y, Dixon S, Pearce M. A survey of music recommendation systems and future perspectives, alongside the fast development of computerized music organizes, Authors clarify a significant metadata-based model and two standard music recommender approaches: pleasing filtering and substance-based model. Despite the way that they have increased phenomenal ground, their weights, for example, perceptible quality propensity and human efforts are clearly self-evident.

Content-based strategies gives recommendations based on the closeness of two melody substance or qualities while collaborative techniques make a forecast on possible inclinations utilizing a framework with appraisals on various songs. Content-based techniques are computationally quick and interpretable. Also, they can be versatile and proficiently adjusted to new things or users. Initially, remove features out of the substance of the tune depictions to make an item portrayal. Second, characterize a similitude work among these article portrayals which imitates what human comprehends as a thing closeness.

In a research paper [4] by Van den Oord A, Dieleman S Schrauwen B, Deep content-based music recommendation. In this paper, authors have explored the utilization of profound convolutional neural networks to foresee dormant elements from music sound when they can't be gotten from use information. Authors assessed the predictions by utilizing them for music suggestion on a modern scale dataset. Despite the fact that a lot of attributes of tunes that influence client inclination can't be anticipated from sound signs, the resulting suggestions appear to be to be reasonable. Authors can infer that foreseeing inert components from music sound is a practical strategy for suggesting new and disagreeable music.

4.

DESIGN AND METHODOLOGY

Data

All metadata and features for all tracks are distributed in fma_metadata.zip (342 MiB). The below tables are used with pandas, a data analysis tool.

- tracks.csv: per track metadata such as ID, title, artist, genres, tags and play counts, for all 106,574 tracks.
- genres.csv: all 163 genres with name and parent (used to infer the genre hierarchy and top-level genres).
- features.csv: common features extracted with librosa.
- echonest.csv: audio features provided by Echonest (now Spotify) for a subset of 13,129 tracks

Nearest neighborhood model

Nearest Neighbourhood model involves collecting information from many users and then making predictions based on some similarity measures between users and between items. This can be classified into user-based and item based models.

In item-based model, it is assumed that songs that are often listened together by some users tend to be similar and are more likely to be listened together in future also by some other user. According to user based similarity model, users who have similar listening histories, i.e., have listened to the same songs in the past tend to have similar interests and will probably listen to the same songs in future too.

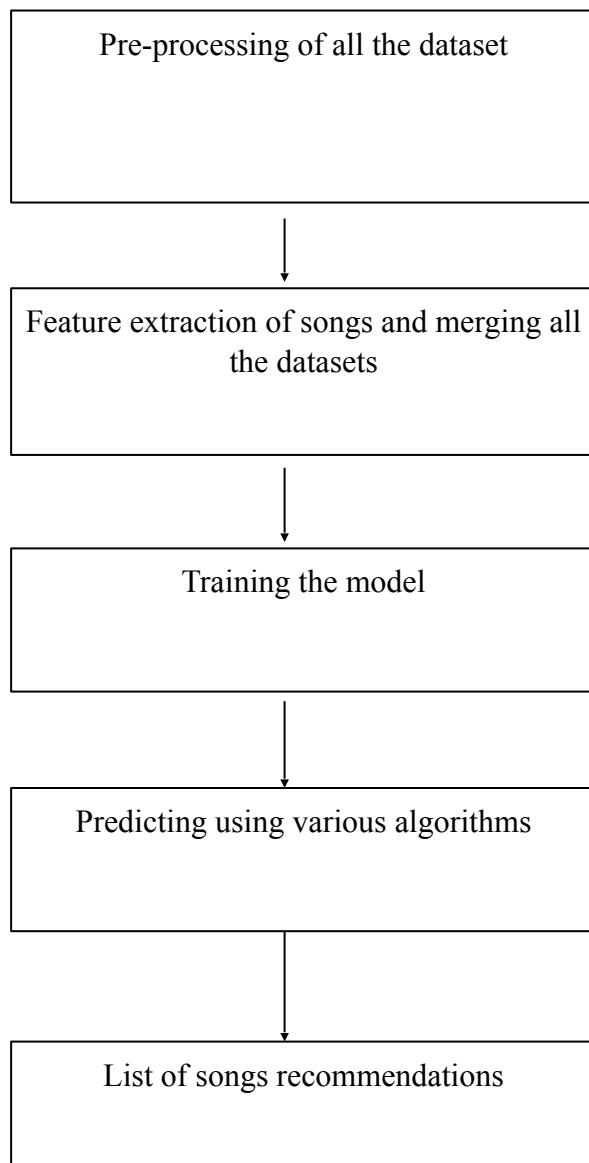
We need some similarity measure to compare between two songs or between two users. Cosine similarity weighs each of the users equally which is usually not the case. User should be weighed less if he has shown interests to many variety of items (it shows that either she does not discern between songs based on their quality, or just likes to explore). Likewise, user is weighted more if listens to very limited set of songs. The similarity measure, $w_{ij} = P(i/j)$, also has drawbacks that some songs which are listened more by users have higher similarity values not because they are similar and listened together but because they are more popular.

Birch Clustering

Existing data clustering methods do not adequately address the problem of processing large datasets with a limited amount of resources (i.e. memory and cpu cycles). In consequence, as the dataset size increases, they scale poorly in terms of running time, and result quality. At a high level, Balanced Iterative Reducing and Clustering using Hierarchies, or BIRCH for short, deals with large datasets by first generating a more compact summary that retains as much distribution information as possible, and then clustering the data summary instead of the original dataset.

BIRCH actually complements other clustering algorithms by virtue of the fact that different clustering algorithms can be applied to the summary produced by BIRCH. BIRCH can only deal with metric attributes (similar to the kind of features KMEANS can handle). A metric attribute is one whose values can be represented by explicit coordinates in an Euclidean space (no categorical variables).

FLOWCHART



5. IMPLEMENTATION (OUTPUT)

1. K- Means

Artist Name based recommendations-

```
In [35]: artist_name_recommend['artist_name'].value_counts()
```

```
Out[35]: 51%      52
         Name: artist_name, dtype: int64
```

```
In [32]: # Artist wise recommendations
         artist_name_recommend.head()
```

```
Out[32]:
```

	album_title	artist_name	genre	track_title
track_id				
34660	Zehu	51%	AvantGarde International Blues Jazz	Hadri Ha'Kat
34661	Zehu	51%	AvantGarde International Blues Jazz	Blender Tzivoni
34662	Zehu	51%	AvantGarde International Blues Jazz	Naniah
34663	Zehu	51%	AvantGarde International Blues Jazz	Yoter Miday
34664	Zehu	51%	AvantGarde International Blues Jazz	Yamim, Lielot

Genre based recommendations-

```
In [36]: genre_recommend['genre'].value_counts()
```

```
Out[36]: Rock      3892
         Name: genre, dtype: int64
```

```
In [25]: # Genre wise recommendations
genre_recommend.head()
```

```
Out[25]:
```

	album_title	artist_name	genre	track_title
track_id				
153	Arc and Sender	Arc and Sender	Rock	Hundred-Year Flood
154	Arc and Sender	Arc and Sender	Rock	Squares And Circles
155	unreleased demo	Arc and Sender	Rock	Maps of the Stars Homes
169	Boss of Goth	Argumentix	Rock	Boss of Goth
170	Nightmarcher	Argumentix	Rock	Industry Standard Massacre

Mixed recommendations-

```
In [37]: genre_recommend['artist_name'].value_counts()
```

```
Out[37]: Glove Compartment      65
Blah Blah Blah                  62
Mors Ontologica                  50
Les Baudouins Morts             38
Kraus                           35
..
Wetnurse                        1
El Sagrado                      1
Colin L. Orchestra              1
So So Glos                      1
Soko                           1
Name: artist_name, Length: 725, dtype: int64
```

```
In [33]: # Mixed Recommendations
mixed_recommend.head()
```

```
Out[33]:
```

	album_title	artist_name	genre	track_title
track_id				
10954	Dig Deep	Just Plain Ant	HipHop	End of the Day (feat. Jay Slim)
12530	netBloc Vol. 13: Color in a world of monochrome	You Are My Everything	Pop	Stalker
10121	Lavori Domestici vol. 2	lucamoka	Electronic	tonesia
15865	Come Fly With Me	The Kid Daytona	HipHop	The Minimum feat. Tiara Wiles (prod. 6th Sense)
11696	Beethoven's Sonata No. 20 in G Major	Daniel Veesev	Classical	Sonata No. 20 in G Major, Op. 49 No. 2 - I. AL...

2. Mini-Batch K-Means

Artist Name based recommendations-

```
In [66]: # Artist wise recommendations
artist_name_recommend_mini.head()
```

```
Out[66]:
```

	album_title	artist_name	genre	track_title
track_id				
34660	Zehu	51%	AvantGarde International Blues Jazz	Hadri Ha'Kat
34661	Zehu	51%	AvantGarde International Blues Jazz	Blender Tzivoni
34662	Zehu	51%	AvantGarde International Blues Jazz	Naniah
34663	Zehu	51%	AvantGarde International Blues Jazz	Yoter Miday
34664	Zehu	51%	AvantGarde International Blues Jazz	Yamim, Lielot

Genre based recommendations-

```
In [65]: # Genre wise recommendations
genre_recommend_mini.head()
```

```
Out[65]:
```

	album_title	artist_name	genre	track_title
track_id				
153	Arc and Sender	Arc and Sender	Rock	Hundred-Year Flood
154	Arc and Sender	Arc and Sender	Rock	Squares And Circles
155	unreleased demo	Arc and Sender	Rock	Maps of the Stars Homes
169	Boss of Goth	Argumentix	Rock	Boss of Goth
170	Nightmarcher	Argumentix	Rock	Industry Standard Massacre

Mixed recommendations-

```
In [67]: # Mixed Recommendations
mixed_mini.head()
```

```
Out[67]:
```

	album_title	artist_name	genre	track_title
track_id				
14543	Everybody Loves the Sun EP	Opio	HipHop	Cherished
21709		Rohr Tarran the Tailor	AvantGarde International	Shameless Hey
11608	Hooray	OCDJ	Electronic	Big Time Stuff
14729	Barriers	pandoras.box	Rock	Minerals
19758	Dirty Mittens Live at PDX Pop Now! '09	Dirty Mittens	Pop	Row

3. Birch Clustering

Genre based recommendations-

```
In [84]: # Genre wise recommendations
genre_recommend_birch.head()
```

```
Out[84]:
```

	album_title	artist_name	genre	track_title
track_id				
153	Arc and Sender	Arc and Sender	Rock	Hundred-Year Flood
154	Arc and Sender	Arc and Sender	Rock	Squares And Circles
155	unreleased demo	Arc and Sender	Rock	Maps of the Stars Homes
169	Boss of Goth	Argumentix	Rock	Boss of Goth
170	Nightmarcher	Argumentix	Rock	Industry Standard Massacre

Artist Name based recommendations-

```
In [85]: # Artist wise recommendations
artist_name_recommend_birch.head()
```

```
Out[85]:
```

	album_title	artist_name	genre	track_title
track_id				
34660	Zehu	51% AvantGarde International Blues Jazz		Hadri Ha'Kat
34661	Zehu	51% AvantGarde International Blues Jazz		Blender Tzivoni
34662	Zehu	51% AvantGarde International Blues Jazz		Naniah
34663	Zehu	51% AvantGarde International Blues Jazz		Yoter Miday
34664	Zehu	51% AvantGarde International Blues Jazz		Yamim, Lielot

Mixed recommendations-

```
In [86]: # Mixed Recommendations
mixed_birch.head()
```

```
Out[86]:
```

	album_title	artist_name	genre	track_title
track_id				
14543	Everybody Loves the Sun EP	Opio	HipHop	Cherished
21709		Rohr Tarran the Tailor	AvantGarde International	Shameless Hey
11608	Hooray	OCDJ	Electronic	Big Time Stuff
14729	Barriers	pandoras.box	Rock	Minerals
19758	Dirty Mittens Live at PDX Pop Now! '09	Dirty Mittens	Pop	Row

6. CONCLUSION AND FUTURE SCOPE

1. Conclusion

Music Recommendation System is used to recommend songs based on factors that have lyrics similarity between songs, audio features of songs, metadata of songs using KNN and Birch algorithm. The system allows users to create playlists, add songs to it and stream it whenever they are logged in. Recommendations are also made based on the same artist. The system contains 2215 songs which can be played. After performing several iterations, the outcome of the result is the different clusters that contain the recommended songs. Every cluster can have different songs depending on the algorithm training in the system. This proposed system will generate recommendations that can be useful for public places mainly restaurants so that songs can be easily managed based on the musical features extracted.

7.2 Future Scope

The future research direction will be mainly focused on user-centric music recommender systems. A survey among athletes showed practitioners in sport and exercise environments tend to select music in a rather arbitrary manner without full consideration of its motivational characteristics. Therefore, future music recommender should be able to lead the users reasonably choose music.

In the future, we would like to try the following things:

1. Using audio signal (e.g. audio frequency) to recommend songs
2. Trying content based algorithm
3. Trying Convolutional Neural Network
4. Making the recommender system a real-time system
5. Trying clustering techniques to recommend music

8. PROJECT CODE

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt import seaborn as sns
import math
import missingno as ms %matplotlib inline

pd.set_option('display.max_rows', 200) pd.set_option('display.max_columns', 2000)

echonest = pd.read_csv('/Users/khushikhandelwal/fma_metadata/echonest.csv' features =
pd.read_csv('/Users/khushikhandelwal/fma_metadata/features.csv' genres = pd.read_csv('/Users/
khushikhandelwal/fma_metadata/genres.csv') tracks = pd.read_csv('/Users/khushikhandelwal/
fma_metadata/tracks.csv')

echonest.info()
features.info()
genres.info()
tracks.info()
echonest.head(10)
ms.matrix(echonest)
echonest.drop(['echonest.8', 'echonest.9', 'echonest.15', 'echonest.16', 'echonest.17', 'echonest.18',
'echonest.19'], axis=1, inplace=True)
echonest.tail(15)
ms.matrix(echonest.iloc[:, 0:15])
echonest.drop(['echonest.10', 'echonest.11', 'echonest.12'], axis=1, inplace=True)
ms.matrix(echonest)
chonest.head(10)
echonest.drop(0, axis=0, inplace=True)
echonest.iloc[0, 0]
echonest.iloc[1, 0]
echonest.iloc[0, 0] = echonest.iloc[1, 0]
echonest.head()
echonest.drop(2, axis=0, inplace=True)
echonest.columns = echonest.iloc[0]
echonest.head()
echonest.drop(1, axis=0, inplace=True)
echonest.head()
echonest.reset_index(inplace=True)
echonest.drop('index', inplace=True, axis=1)
echonest.head()
type(echonest['acousticness'])[0])
def convert_to_float(df, columns):
```

```

    for i in columns:
        df[i] = df[i].astype('float')
    return df
echonest = convert_to_float(echonest, set(echonest.columns) - set(['track_id', 'artist_name',
'release']))
echonest.head()
echonest.info()
features.info()
features.head(10)
ms.matrix(features.iloc[:, 21:40])
features.iloc[0,0] = features.iloc[2, 0]
features.head(3)
features.drop(2, inplace=True)
len(features.columns)
def combine_two_rows(df):
    columns = list(df.columns)
    for i in range(0, 519):
        columns[i] = columns[i] + " " + df.iloc[0, i]
    return columns
features.columns = combine_two_rows(features)
features.drop([0, 1], inplace=True)
features.reset_index(inplace=True)
features.drop('index', axis=1, inplace=True)
features.head()
features = features.astype(dtype='float')
features['feature track_id'] = features['feature track_id'].astype('int')
ms.matrix(features)
features.head(3)
tracks.info()
tracks.head()
tracks.iloc[0,0] = tracks.iloc[1, 0]
tracks.drop(1, axis=0, inplace=True)
tracks.head()
len(tracks.columns)
def combine_one_row(df):
    columns = list(df.columns)
    for i in range(0, 53):
        if i == 0:
            columns[i] = df.iloc[0, i]
        else:
            columns[i] = columns[i] + " " + df.iloc[0, i]
    return columns
tracks.columns = combine_one_row(tracks)

```

```

tracks.drop(0, inplace=True)
tracks.reset_index(inplace=True)
tracks.drop(['index'], axis=1, inplace=True)
ms.matrix(tracks.iloc[0: 10])
tracks.head()
tracks['track.7 genre_top'].value_counts()
track_title = pd.DataFrame(tracks['track.19 title'])
track_title['track_id'] = tracks['track_id']
track_title.head()
track_title.tail()
track_title.shape
tracks.drop(['album comments', 'album.1 date_created',
            'album.2 date_released', 'album.11 tracks',
            'album.9 tags', 'album.8 producer', 'album.3 engineer', 'album.6 information',
            'artist active_year_begin', 'artist.1 active_year_end', 'artist.2 associated_labels',
            'artist.3 bio', 'artist.4 comments', 'artist.5 date_created', 'artist.7 id',
            'artist.8 latitude', 'artist.9 location', 'artist.10 longitude', 'artist.11 members',
            'artist.13 related_projects', 'artist.14 tags', 'artist.15 website', 'artist.16 wikipedia_page',
            'set.1 subset', 'track.1 comments', 'track.2 composer', 'track.3 date_created', 'track.4
date_recorded',
            'track.10 information', 'track.13 license', 'track.15 lyricist', 'track.17 publisher', 'track.18
tags',
            'track.19 title'], axis=1, inplace=True)
tracks.info()
ms.matrix(tracks)
tracks['album.12 type'].value_counts()
tracks['album.10 title'].value_counts()
tracks['album.10 title'].fillna(method='ffill', inplace=True)
tracks.drop(['track.12 language_code', 'album.12 type'], axis=1, inplace=True)
tracks.drop('track.9 genres_all', axis=1, inplace=True)
ms.matrix(tracks)
tracks['track.8 genres'].unique()
genres.info()
type(tracks['track.7 genre_top'].iloc[27])
def getList(cd):
    return cd[1:-1].split(',')
for i in range(0, 106574):
    if type(tracks['track.7 genre_top'][i]) == float:
genre_list = getList(str(tracks['track.8 genres'][i]))
    count = len(genre_list)
    title = ""
    for j in range(0, count):
        title = title + str(genres['title'][j]) + str(',')

```



```

tracks['track.7 genre_top'][i] = title
tracks['track.7 genre_top'].head()
genres.info()
ms.matrix(genres)
genres.head()
echonest.info()
tracks.info()
tracks.head()
echonest.head()
genres.info()
features.info()
features.columns = ['track_id'] + list(features.columns[1:])
features.head()
type(echonest['track_id'].iloc[0])
echonest['track_id'] = echonest['track_id'].astype('int')
tracks['track_id'] = tracks['track_id'].astype('int')
features.sort_values(by='track_id', inplace=True)
tracks.sort_values(by='track_id', inplace=True)
echonest.sort_values(by='track_id', inplace=True)
features.head()
tracks.head()
count = 0
for i in range(0, 106574):
    if features['track_id'][i] == tracks['track_id'][i]:
        count += 1
    else:
        print(features['track_id'][i], tracks['track_id'][i])
final = pd.concat([features, tracks.drop('track_id', axis=1)], axis=1)
final.shape
final.head()
echonest.tail(3)
echonest.drop(['artist_name', 'release'], axis=1, inplace=True)
tracks.tail(3)
features.head(1)
final = echonest.merge(final, on='track_id')
final.shape
ms.matrix(final)
final.head()
final.shape
final.shapefinal.shape
final['track.7 genre_top'].value_counts()
def format_strings(x):
    if '-' in x:

```

```

        return ".join(x.split('-'))
    if x.find('/'):
        return "|".join(x.split('/'))
    return x
def modifyString(serie, val):
    for i in range(0, val):
        if serie[i] == 'Old-Time / Historic':
            serie[i] = 'OldTime|Historic'
    return serie
final['track.7 genre_top'] = modifyString(final['track.7 genre_top'], 13129)
final['track.7 genre_top'] = final['track.7 genre_top'].apply(format_strings)
final['track.7 genre_top'].value_counts()
final.head()
metadata = pd.DataFrame()
metadata['track_id'] = final['track_id']
metadata.shape
track_title.shape
track_title = track_title.set_index('track_id')
track_title.head()
track_title.index = [int(i) for i in track_title.index]
track_title.head()
metadata.head()
metadata['album_title'] = final['album.10 title']
metadata['artist_name'] = final['artist.12 name']
metadata['genre'] = final['track.7 genre_top']
metadata = metadata.set_index('track_id')
metadata.tail()
metadata.head()
metadata['track_title'] = track_title.loc[metadata.index]['track.19 title']
metadata.tail()

metadata.head()
len(metadata[metadata['genre'].isnull()])
final.drop('album.10 title', axis=1, inplace=True)
final.head()

final.info()

final.drop('artist.12 name', axis=1, inplace=True)

final.info()

final.head()

```

```

k = final

final.head()

final.drop('set split', axis=1, inplace=True)

final.info()

final.info()

genres['title'].count()

genre_dummy = pd.DataFrame(data= np.zeros((13129, 163)), columns=
list(genres['title'].unique()))

genre_dummy.head()
genre_list = pd.Series(data= genre_dummy.columns)

genre_list = modifyString(genre_list, 163)

genre_list = genre_list.apply(format_strings)

genre_dummy.columns= genre_list

# columns converted successfully

genre_list = list(genre_list)

final

for i in range(0, 13129):
    if '|' in final['track.7 genre_top'][i]:
        divided_list = str(final['track.7 genre_top'][i]).split('|')
        count = len(divided_list)
        for j in range(0, count):
            if divided_list[j] in genre_list:
                location = genre_list.index(divided_list[j])
            genre_dummy.iloc[i, location] = 1
        else:
            location = genre_list.index(final['track.7 genre_top'][i])
            genre_dummy.iloc[i, location] = 1

```

```

genre_list.index(final['track.7 genre_top'])[0])

final.drop(['track.7 genre_top'], axis= 1, inplace= True)
final = pd.concat([final, genre_dummy], axis= 1)

final.head()

import os

if not os.path.isdir(os.path.join('datasets','final')):
os.makedirs(os.path.join('datasets','final'))

metadata.to_csv('datasets/final/metadata.csv')
final.to_csv('datasets/final/final.csv')
import pandas as pd
import numpy as np

final = pd.read_csv('/Users/khushikhandelwal/datasets/final/final.csv') metadata = pd.read_csv('/
Users/khushikhandelwal/datasets/final/metadata.csv')

final.head()

metadata.head()

#Model Selection - K Means Algorithm

In [7]:fromsklearn.cluster import KMeans from sklearn.utils import shuffle

final = shuffle(final)

X = final.loc[[i for i in range(0, 6000)]] Y = final.loc[[i for i in range(6000, final.shape[0])]]

X = shuffle(X) Y = shuffle(Y)

metadata.head()

metadata = metadata.set_index('track_id')

kmeans = KMeans(n_clusters=6)

Y.head()

```

```

def fit(df, algo, flag=0): if flag: algo.fit(df) else: algo.partial_fit(df) df['label'] = algo.labels_
return (df, algo)

def predict(t, Y): y_pred = t[1].predict(Y) mode = pd.Series(y_pred).mode() return t[0][t[0]
['label'] == mode.loc[0]]

def recommend(recommendations, meta, Y): dat = [] for i in Y['track_id']: dat.append(i)
genre_mode = meta.loc[dat]['genre'].mode() artist_mode = meta.loc[dat]['artist_name'].mode()
return meta[meta['genre'] == genre_mode.iloc[0]], meta[meta['artist_name'] ==
artist_mode.iloc[0]], meta.loc[recommendations['track_id']]

t = fit(X, kmeans, 1)

recommendations = predict(t, Y)

output = recommend(recommendations, metadata, Y)

genre_recommend, artist_name_recommend, mixed_recommend = output[0], output[1],
output[2]

genre_recommend.shape

artist_name_recommend.shape

mixed_recommend.shape

# Genre wise recommendations
genre_recommend.head()

# Artist wise recommendations
artist_name_recommend.head()

# Mixed Recommendations
mixed_recommend.head()

recommendations.head()

artist_name_recommend['artist_name'].value_counts()

genre_recommend['genre'].value_counts()

genre_recommend['artist_name'].value_counts()

```

```

#Testing
testing = Y.iloc[6:12]['track_id']

testing

ids = testing.loc[testing.index]

songs = metadata.loc[testing.loc[list(testing.index)]]

songs

re = predict(t, Y.iloc[6:12])

output = recommend(re, metadata, Y.iloc[6:12])

ge_re, ge_ar, ge_mix = output[0], output[1], output[2]

ge_re.head()

ge_ar.head(10)

ge_mix.head(10)

ge_re.shape

ge_ar.shape

ge_mix.shape

#Model Selection - MiniBatchKMeans

from sklearn.cluster import MiniBatchKMeans

mini = MiniBatchKMeans(n_clusters = 6)

X.drop('label', axis=1, inplace=True)

part_1, part_2, part_3 = X.iloc[0: 2000], X.iloc[2000:4000], X.iloc[4000:6000]

for i in [part_1, part_2, part_3]: t = fit(i, mini) mini = t[1] i = t[0]

X = pd.concat([part_1, part_2, part_3])

```

```

X.columns

X.head(3)

X['label'].value_counts()

recommendations = predict((X, mini), Y)

output = recommend(recommendations, metadata, Y)

genre_recommend_mini, artist_name_recommend_mini, mixed_mini = output[0], output[1],
output[2]

genre_recommend_mini.shape

artist_name_recommend_mini.shape

# Genre wise recommendations
genre_recommend_mini.head()

# Artist wise recommendations
artist_name_recommend_mini.head()

# Mixed Recommendations
mixed_mini.head()

genre_recommend_mini['artist_name'].value_counts()

#Model Selection - Birch

from sklearn.cluster import Birch

birch = Birch(n_clusters = 6)

X.drop('label', axis=1, inplace=True)

part_1, part_2, part_3 = X.iloc[0: 2000], X.iloc[2000:4000], X.iloc[4000:6000]

for i in [part_1, part_2, part_3]: t = fit(i, birch) mini = t[1] i = t[0]

X = pd.concat([part_1, part_2, part_3])

X.columns

```

```

X.head(3)

X['label'].value_counts()

recommendations = predict((X, birch), Y)

output = recommend(recommendations, metadata, Y)

genre_recommend_birch, artist_name_recommend_birch, mixed_birch = output[0], output[1],
output[2]

genre_recommend_birch.shape

artist_name_recommend_birch.shape

# Genre wise recommendations
genre_recommend_birch.head()

# Artist wise recommendations
artist_name_recommend_birch.head()

# Mixed Recommendations
mixed_birch.head()

```

References

- 1] Bertin-Mahieux, T., Ellis, D.P., Whitman, B. and Lamere, P., 2011. The million song dataset.
- [2] Song Y, Dixon S, Pearce M. A survey of music recommendation systems and future perspectives. In 9th International Symposium on Computer Music Modeling and Retrieval 2012 Jun 19 (Vol. 4, pp. 395-410).
- [3] EugeInzaugarat. "The ABC of building a content-based music recommender system." Towards DataScience. <https://towardsdatascience.com/the-abc-of-building-a-music-recommender-system-part-i-230e99da9cad> (accessed Feb 19, 2020).
- [4] Van den Oord A, Dieleman S, Schrauwen B. Deep content-based music recommendation. In Advances in neural information processing systems 2013 (pp. 2643-2651).

- [5] Zhang, Muhan, and Yixin Chen. "Inductive matrix completion based on graph neural networks." arXiv preprint arXiv:1904.12058 (2019).
- [6] Rao N, Yu HF, Ravikumar PK, Dhillon IS. Collaborative filtering with graph information: Consistency and scalable methods. In Advances in neural information processing systems 2015 (pp. 2107-2115).